# Agri Vision
# Alternatives to YOLO

**Architecture selection for federated crop-disease detection**

Prepared for supervisor discussion • 5 October 2026

## Recommendation at a glance

For an accuracy-focused comparison in this FYP, I recommend Faster R-CNN with a ResNet50 backbone and a Feature Pyramid Network (FPN). It gives us a well-established two-stage detector to compare with the existing one-stage model, and it can be implemented within the project's PyTorch environment. The recommendation concerns the value of the experiment; it does not claim that Faster R-CNN has already outperformed the current model.

If the main requirement is a small model for resource-limited clients or eventual mobile use, SSDLite with MobileNetV3-Large is the more suitable alternative. These are different priorities, so a single architecture should not be called "best" without saying what we are optimising for.

| Decision | Recommended choice |
|---|---|
| **Alternative for an accuracy-focused academic comparison** | Faster R-CNN + ResNet50-FPN |
| **Alternative when client resources and communication dominate** | SSDLite320 + MobileNetV3-Large |
| **Other candidates for a longer study** | EfficientDet-D0 and RT-DETR |
| **Submission strategy with 15 days remaining** | Keep the working baseline; attempt one bounded comparison only if core deliverables are on schedule. |

## What this document establishes

The project's measured results, published architectural features, and proposed next steps are distinguished throughout. No non-YOLO model has been trained or benchmarked on Agri Vision for this review. Expected benefits are hypotheses to test. Published COCO results are included only where clearly labelled and are not forecasts of crop-disease accuracy.

The practical recommendation is to obtain a defensible comparison without making final submission depend on an untested replacement. A reliable demonstration and a clear account of limitations remain essential parts of the FYP.

# 1. What Agri Vision needs from a detector

Agri Vision locates and labels crop-disease or pest-related objects using bounding boxes. A whole-image classifier alone would not preserve this function. The project currently uses 30 classes spanning banana, cauliflower, chili, groundnut, and radish. Healthy categories are also included. [P1]

| Project characteristic | Implication for architecture selection |
|---|---|
| **15,512 training; 4,374 validation; 2,191 test images** | Use transfer learning and preserve the existing evaluation split. |
| **Three clients, each starting a round from identical global weights** | All clients need the same architecture and identical label ordering. |
| **Sample-weighted FedAvg; clients simulated sequentially on one GPU** | Measure compute and model size. Real network performance has not been demonstrated. |
| **Tesla T4 used for the recorded Colab run** | GPU memory and available session time constrain training. |
| **15 days until submission** | Implementation effort and failure recovery matter alongside detection scores. |

## Current evidence

| Round-35 test evaluation | Precision | Recall | mAP50 | mAP50–95 |
|---|---|---|---|---|
| **All 30 classes** | 63.72% | 54.81% | 55.72% | 37.26% |
| **24 non-chili classes** | 69.81% | 62.73% | 65.16% | 43.44% |
| **Six chili classes** | 39.37% | 23.14% | 17.94% | 12.53% |

These are saved test results at image size 640. The non-chili row is a class-average subset of the same trained model, with all test images retained. It is not the outcome of retraining without chili. Precision and recall use the operating point selected for the full model. [P2]

In rounds 26–35, the highest saved validation mAP50 was 61.22% at round 33; the highest validation mAP50–95 was 40.61% at round 30. These validation summaries average client scores and should not be directly compared with the test rows above. The late decline shows why changing architecture is only one possible intervention. Labels, data diversity, and the training schedule also need attention. [P3]

# 2. Primary recommendation: Faster R-CNN

## How it works

Faster R-CNN first identifies promising regions and then classifies and refines them. A Region Proposal Network shares image features with the detector, so proposals are learned as part of the system. In the proposed configuration, ResNet50 extracts visual features and FPN supplies representations at several scales. [1, 2]

| Image | ResNet50 + FPN | Region proposals | Classification and box refinement |
|---|---|---|---|
| **Crop photograph** | Features at different scales | Candidate object regions | Disease/pest label, box and confidence |

## Why it is the strongest comparison for this FYP

First, it changes the detection approach in a meaningful way. The study can ask whether a proposal-based detector offers a worthwhile improvement over the existing one-stage baseline, rather than comparing two closely related model sizes.

Second, the model provides a sensible way to investigate localisation. Agri Vision contains both large leaves and smaller annotated objects. FPN makes multi-scale features available to the detector. This motivates testing difficult cases; it does not establish that small lesions or insects will automatically be detected better. Their pixel size and annotation quality still matter. [2]

Third, there is a documented TorchVision implementation and a custom-dataset fine-tuning example. The project already uses PyTorch, so the surrounding data and federated workflow can be adapted without introducing a different deep-learning framework. [3, 9]

Fourth, a comparison remains useful even if the new detector is slower or less accurate. It allows the final report to justify the tradeoff between detection quality, inference time, and the cost of exchanging model parameters.

## Where it is a weaker fit

The standard ResNet50-FPN model is considerably larger than the current nano detector and the SSDLite alternative. Its resource cost is a real concern when training three clients repeatedly. The proposed role is therefore a controlled comparison, not an immediate replacement of the working application. Actual recall, mAP, memory use, and latency on this dataset remain unmeasured.

A new experiment must start from suitable pretrained Faster R-CNN weights with a new Agri Vision prediction head. The existing YOLO round-35 checkpoint cannot resume training in this architecture because the layers and tensor shapes differ.

# 3. Other non-YOLO options

| Candidate | Best reason to consider it | Main concern for Agri Vision |
|---|---|---|
| **SSDLite + MobileNetV3** | A compact mobile-oriented detector and an efficiency comparison. | The standard 320 × 320 input may discard useful detail in small objects. |
| **EfficientDet-D0** | A compact multi-scale design using EfficientNet and a weighted bidirectional feature pyramid. | A new implementation and training recipe add integration work. |
| **RT-DETR** | A transformer-based, end-to-end detector for a broader research comparison. | More model and training adaptation than is attractive at this deadline. |

## SSDLite with MobileNetV3-Large

SSDLite predicts detections in one stage using an efficient feature extractor. TorchVision provides a pretrained 320 × 320 configuration. It is the preferred option if smaller model updates and resource-limited inference are the supervisor's main priorities. It still requires a new data adapter, detection head, and training loop. [4, 5]

Its compact design does not guarantee better communication efficiency than Agri Vision's existing YOLOv8n: the actual 30-class state size must be measured. A low-resolution input also makes it an uncertain choice when the immediate goal is recovering tiny disease symptoms. If input resolution or feature layers are changed, that becomes a modified model requiring its own validation.

## EfficientDet-D0

EfficientDet combines an EfficientNet backbone, a weighted bidirectional feature pyramid, and a family of scaled detector sizes. D0 is the smallest original configuration and is worth considering for a longer efficiency study. [6]

For this deadline, I would rank it below the TorchVision options. The integration must account for the selected implementation's preprocessing, anchors, losses, checkpoints, and pretrained weights. Moving to a different framework would add further work. Its architectural efficiency alone does not establish superior federated behaviour.

## RT-DETR

RT-DETR uses an efficient hybrid encoder and a transformer decoder to predict object sets, avoiding the usual non-maximum-suppression stage. It offers a different research direction and published real-time detection results, but those results use specified benchmark conditions. [7]

I would retain it as future work unless a working implementation and adequate GPU time are already available. Real-time inference claims do not mean that training the model is inexpensive or that its crop-disease recall will be higher.

# 4. Resource and federated-learning tradeoffs

A detector is trained repeatedly in federated learning, and its parameters are exchanged after local updates. Larger models increase the amount of data involved when full parameters are transmitted. This cost should be discussed alongside detection quality. FedAvg combines local updates weighted by the amount of participating training data. [8]

| Published reference configuration | Parameters | COCO box AP | FP32 parameter payload* |
|---|---|---|---|
| **Faster R-CNN ResNet50-FPN, COCO_V1** | 41.76 million | 37.0 | 167.0 MB |
| **SSDLite320 MobileNetV3-Large, COCO_V1** | 3.44 million | 21.3 | 13.8 MB |

*Illustrative payload = published parameter count × 4 bytes; MB uses 1,000,000 bytes. The figures describe stock pretrained models. A 30-class head changes the count. Buffers, transport overhead, precision, compression, and frozen-parameter policy affect actual transfers. These are not measured Agri Vision communication costs. [3, 4]

The COCO AP entries are context, not a controlled Agri Vision comparison. Input sizes and training recipes differ. Neither the AP values nor any published FPS should be presented as performance achieved by this project.

## What a three-client round implies

With one full-model download and one upload per client, three clients exchange approximately six parameter payloads per round. Using the illustrative sizes above, that is about 1.00 GB for Faster R-CNN and 82.6 MB for SSDLite. Agri Vision currently simulates clients in one Colab process, so this is an estimate for a future distributed deployment, not traffic observed in the existing run.

## Implementation decisions that affect fairness

Every client must start from the same global weights and use the same ordered parameter structure. The prediction head must have identical dimensions on every client. Non-floating counters and normalization buffers need an explicit policy; blindly averaging every state entry is inappropriate.

Freezing the backbone can shorten a pilot, but it changes the experiment. Report which layers were trained and whether frozen weights were transmitted. Similarly, a compact model should not be declared better solely because it finished more updates within the time limit: report both the training budget and the resulting scores.

Architectural choice does not itself guarantee privacy. The current prototype keeps raw images within logical client partitions, but parameter sharing and a single-machine simulation should not be described as a proven secure deployment.

# 5. What would change in the project

The existing dataset partitioning and high-level FedAvg round structure can be reused. The model construction, local training, prediction adapter, and evaluation path need adaptation. A separate experimental entry point would keep the completed baseline reproducible.

| Work item | Required adaptation |
|---|---|
| **Dataset adapter** | Read existing images and YOLO labels; convert normalized centre-width-height boxes into pixel corner coordinates. |
| **Label mapping** | For the proposed TorchVision detector, reserve label 0 for background and map Agri Vision IDs 0–29 to 1–30. |
| **Model head** | Load pretrained Faster R-CNN, then replace its box predictor for 31 outputs: 30 project categories plus background. |
| **Local trainer** | Supply image lists and target dictionaries, sum the returned detection losses, and run optimizer updates. |
| **Federated interface** | Exchange the chosen tensors in a stable order, verify shapes, and aggregate using actual training-image counts. |
| **Evaluation and app** | Convert predictions to the existing class IDs and original-image coordinates; produce common metrics and display boxes. |

For an image of width W and height H, convert a YOLO annotation (xc, yc, w, h) as x1 = (xc − w/2)W, y1 = (yc − h/2)H, x2 = (xc + w/2)W, and y2 = (yc + h/2)H. Clip boxes to image bounds, reject degenerate boxes, and apply spatial transformations to images and boxes together. The original label files can remain intact.

TorchVision's fine-tuning tutorial documents replacing a pretrained detection head and using target dictionaries. Loading a stock pretrained detector and merely changing a class-count argument is not a substitute for a correctly adapted head. A matching torch/torchvision build and working compiled detection operators must also be verified in Colab. [9]

## Checks before a full experiment

Start by drawing converted boxes over a small sample of images. Then verify a finite training loss, confirm that parameters change after an update, and reload a saved checkpoint. Test one complete three-client round and check aggregation shapes, counts, and prediction labels. A small-subset overfit check can help expose a broken label or training pipeline before spending a full GPU session.

Architecture experiments should keep separate checkpoints and output folders. The round numbers of a newly initialized Faster R-CNN model do not continue the learning history of the YOLO checkpoint; its training history must be reported separately.

# 6. How to make the comparison convincing

A useful research question is: “Does Faster R-CNN improve crop-disease detection sufficiently to justify its extra computation and model-exchange cost?” That can be answered even if the result favours the existing detector.

| Measure | How to report it |
|---|---|
| **mAP50–95** | Primary detection-quality criterion across multiple box-overlap thresholds. |
| **mAP50** | Complementary measure at IoU 0.50. |
| **Precision and recall** | State the operating-point rule; also compare recall at a predeclared precision target when practical. |
| **Per-class AP and recall** | Include weak classes and class support; an overall average can hide severe failures. |
| **Time and memory** | Record local epoch time, complete round time, peak GPU memory, and end-to-end inference latency. |
| **Model size and transfer** | Count actual exchanged tensors and bytes; label estimates separately from measurements. |

Use the same source images, class definitions, seed-42 client partitions, and held-out test set. Choose checkpoints and settings using validation data. A weighted average of client AP values is not equivalent to AP computed from pooled predictions; use the same aggregation convention for every compared model. If pooled validation is added, recompute it for both models.

Use a stated budget. A practical comparison can report both equal passes through the local data and equal GPU time. Model-specific learning rates may be necessary, but each architecture should receive a comparable tuning allowance. A new detector trained briefly should not be compared with the accumulated 35-round model as if their training budgets were equal.

Preprocessing must be visible in the report. The standard SSDLite configuration uses 320 × 320 inputs, whereas the current detector was evaluated at 640. An “out-of-the-box” comparison can preserve those configurations, but it does not isolate architecture alone. Record each detector’s actual resize policy and input dimensions.

## What would justify adopting the alternative?

Before training, agree on an acceptable change in mAP, weak-class recall, inference latency, and training cost. Adopt the alternative only if the measured tradeoff is useful for the application. Where time permits, repeat the strongest comparison with more than one random seed. If the deadline allows only one run, describe it as an exploratory result rather than proof of consistent superiority.

No architecture removes the need to review ambiguous disease labels, missing objects, or near-duplicate images. The dataset audit remains relevant whichever detector is selected.

# 7. A realistic decision within 15 days

My recommendation is to preserve the completed system as the submission baseline and time-box one non-YOLO pilot. The purpose of the pilot is to establish feasibility and obtain a defensible comparison, not to make final submission depend on an unfinished redesign. The following is a proposed schedule, not a guarantee of training time.

| Time window | Action and decision |
|---|---|
| **Days 1–2** | Confirm application/report progress. If a pilot is approved, build the Faster R-CNN data adapter, test box overlays, and profile one local training run. |
| **Days 3–4** | Complete a federated smoke test and, only if resources permit, a bounded comparison with a declared budget. Freeze architecture changes at the end of day 4. |
| **Days 5–7** | Finalize the selected model's inference workflow, error handling, and demonstration. |
| **Days 8–11** | Complete methodology, experiments, results, limitations, and references. |
| **Days 12–15** | Reproduce setup, resolve issues, prepare slides, rehearse the viva, and retain backups. |

If the new pipeline is not correct and stable by the end of day 2, stop the pilot and keep this architecture review as a proposed extension. If the application or report is significantly unfinished, skip the pilot entirely. A clear feasibility analysis is more useful than unsupported performance claims.

## Suggested explanation to the supervisor

"I propose Faster R-CNN with ResNet50-FPN as the main non-YOLO comparison for Agri Vision. Its region-proposal approach gives us a meaningful alternative for studying detection and localisation, and PyTorch provides a documented implementation. I would evaluate whether any gain in recall or mAP is worth the additional training and communication cost. I am not assuming that it will outperform our current model. If lightweight deployment is the higher priority, I would instead choose SSDLite with MobileNetV3. With 15 days remaining, I would keep the working system as the baseline and limit any new experiment to a clearly defined pilot."

## Decision requested

The supervisor can select one of three scopes: approve the short Faster R-CNN comparison; choose SSDLite for an efficiency-focused comparison; or retain the architecture study as future work and concentrate on finalising the current system. Each is defensible if the project's objective, evidence, and remaining time are stated clearly.

# References and project evidence

Primary papers and official implementation documentation were consulted on 5 October 2026. Reference numbers identify architectural sources; P1–P4 identify local project evidence. Recommendations and schedule estimates are the analysis in this document.

[1] Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.

Original architecture paper

[2] Lin, T.-Y., et al. (2017). Feature Pyramid Networks for Object Detection. CVPR.

Multi-scale feature extraction

[3] PyTorch / TorchVision. fasterrcnn_resnet50_fpn model documentation, COCO_V1 reference configuration.

Implementation and published model statistics

[4] PyTorch / TorchVision. ssdlite320_mobilenet_v3_large model documentation, COCO_V1 reference configuration.

Implementation and published model statistics

[5] PyTorch (2021). Everything You Need To Know About Torchvision’s SSDlite Implementation.

Mobile-oriented detector design and implementation

[6] Tan, M., Pang, R., and Le, Q. V. (2020). EfficientDet: Scalable and Efficient Object Detection. CVPR.

EfficientDet architecture

[7] Zhao, Y., et al. (2024). DETRs Beat YOLOs on Real-time Object Detection. CVPR.

RT-DETR architecture

[8] McMahan, H. B., et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS.

FedAvg and decentralized training

[9] PyTorch. TorchVision Object Detection Finetuning Tutorial.

Custom labels, targets, and prediction-head adaptation

## Local project records

[P1] README.md; model.py; federated_client.py; colab_train.py. Project architecture, data counts, and federated workflow.

[P2] reports/round_035_non_chili_summary.json and reports/round_035_test_per_class.csv. Measured round-35 test results.

[P3] reports/rounds_026_035_global_metrics.csv and reports/rounds_026_035_detailed_report.md. Saved validation history.

[P4] reports/chili_dataset_merge.md. Dataset additions, duplicate checks, and annotation conflicts.